



**TECNOLÓGICO
NACIONAL DE MÉXICO**



**INGENIERÍA EN
SISTEMAS
COMPUTACIONALES**

TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA

DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

SEMESTRE ENERO – AGOSTO 2026

CARRERA

Ing. Sistemas computacionales

MATERIA

Patrones de diseño

TÍTULO

Patrón de diseño estado

NOMBRE Y NÚMERO DE CONTROL DEL ALUMNO

Castillejo Robles Lennyn Alejandro 22210880

NOMBRE DEL MAESTRO

Maribel Guerrero Luis

FECHA DE ENTREGA

04 de mayo del 2026

Introducción

En el desarrollo de sistemas inteligentes, la automatización y la adaptación al entorno son aspectos clave para mejorar la eficiencia y la experiencia del usuario. En este contexto, se diseñó e implementó un simulador de aire acondicionado automático, capaz de detectar y regular la temperatura de manera autónoma, alternando entre modos de enfriamiento y calefacción según las condiciones del ambiente.

Para lograr este comportamiento dinámico, se utilizó el patrón de diseño Estado (State), el cual permite que un objeto modifique su comportamiento cuando cambia su estado interno, facilitando una estructura más organizada, escalable y fácil de mantener. Este enfoque resulta ideal para modelar sistemas como un aire acondicionado, donde las acciones dependen directamente del estado actual, como “enfriando”, “calentando” o “apagado”.

El programa simula el funcionamiento de un sistema de climatización inteligente, tomando decisiones en tiempo real basadas en la temperatura detectada, lo que permite representar de manera clara cómo los patrones de diseño pueden aplicarse a problemas del mundo real. Además, este proyecto demuestra la importancia de una correcta arquitectura de software para desarrollar soluciones eficientes, reutilizables y fáciles de extender.

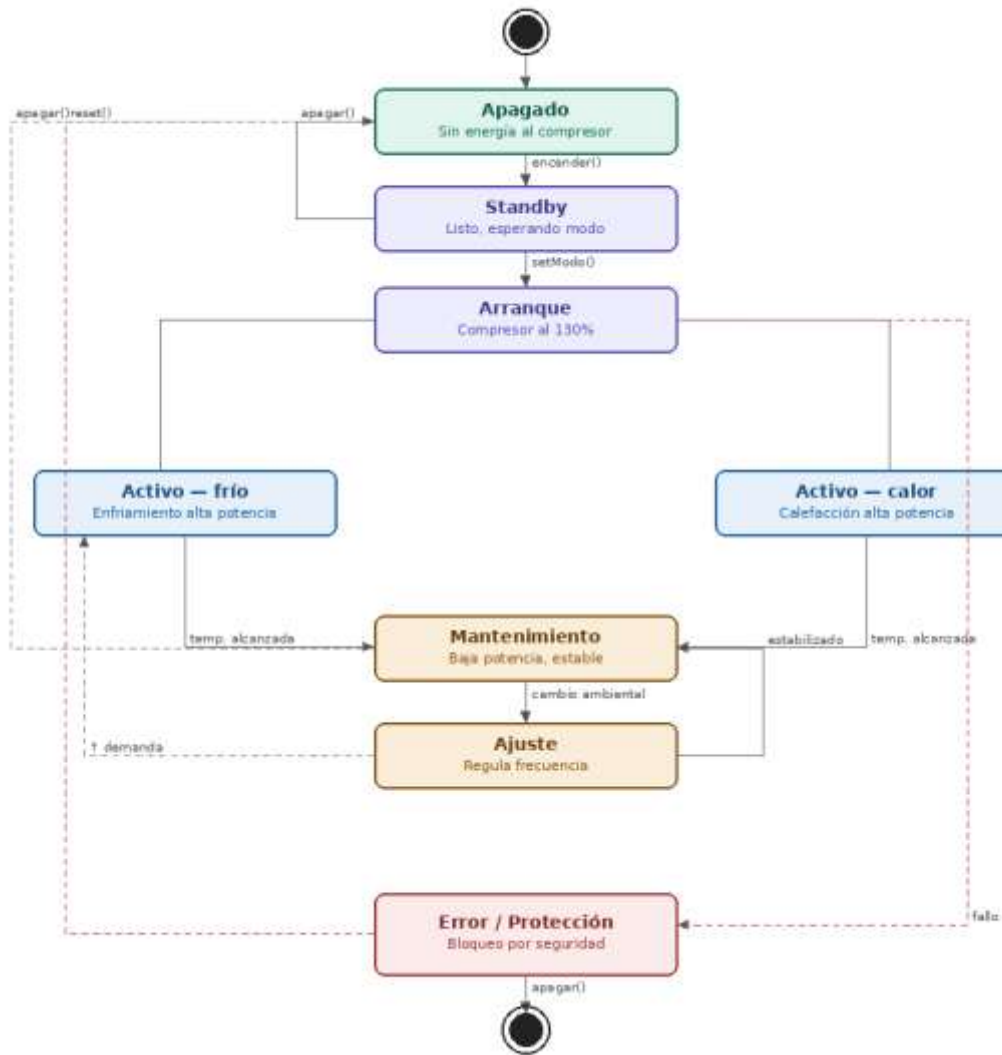
Objetivo General

- Desarrollar un simulador de aire acondicionado automático utilizando el patrón de diseño Estado (State), capaz de detectar y regular la temperatura del ambiente de manera autónoma, alternando entre modos de enfriamiento, calefacción y reposo para mantener condiciones óptimas.

Objetivos Específicos

- Implementar el patrón de diseño Estado para modelar los diferentes comportamientos del sistema según la temperatura detectada.
- Simular el funcionamiento de un aire acondicionado inteligente que responda automáticamente a cambios en el entorno.

Diagrama de estados



Implementación del patrón Estado

```
from typing import Optional

class AireAcondicionado:
    """Contexto del patrón Estado. Guarda temperaturas, modo y estado actual."""

    def __init__(self, temperatura_inicial: float = 28.0):
        self.temperatura_deseada: float = 22.0
        self.temperatura_actual: float = temperatura_inicial
        self.modo: str = "FRIO"
        self._estado: Optional['EstadoAC'] = None

        from estados.estado_apagado import EstadoApagado
        self.set_estado(EstadoApagado(self))

    def set_estado(self, estado: 'EstadoAC') -> None:
        self._estado = estado

    def encender(self) -> None: self._estado.encender()
    def apagar(self) -> None: self._estado.apagar()
    def set_modo(self, modo: str) -> None: self._estado.set_modo(modo)
    def monitorear(self) -> None: self._estado.monitorear()
    def reset(self) -> None: self._estado.reset()

    def simular_cambio_ambiental(self, nueva_temperatura: float) -> None:
        self.temperatura_actual = nueva_temperatura
```

Estados

```
✓ estados
  > __pycache__
    estado_activo_calor.py
    estado_activo_frio.py
    estado_ajuste.py
    estado_apagado.py
    estado_arranque.py
    estado_error.py
    estado_mantenimiento.py
    estado_standby.py
```

Clase abstracta para los estados del AC

```
from abc import ABC, abstractmethod

class EstadoAC(ABC):
    """Clase base abstracta para todos los estados del AC."""

    def __init__(self, ac: 'AireAcondicionado'):
        from aire_acondicionado import AireAcondicionado
        self._ac = ac

    @abstractmethod
    def encender(self) -> None: pass

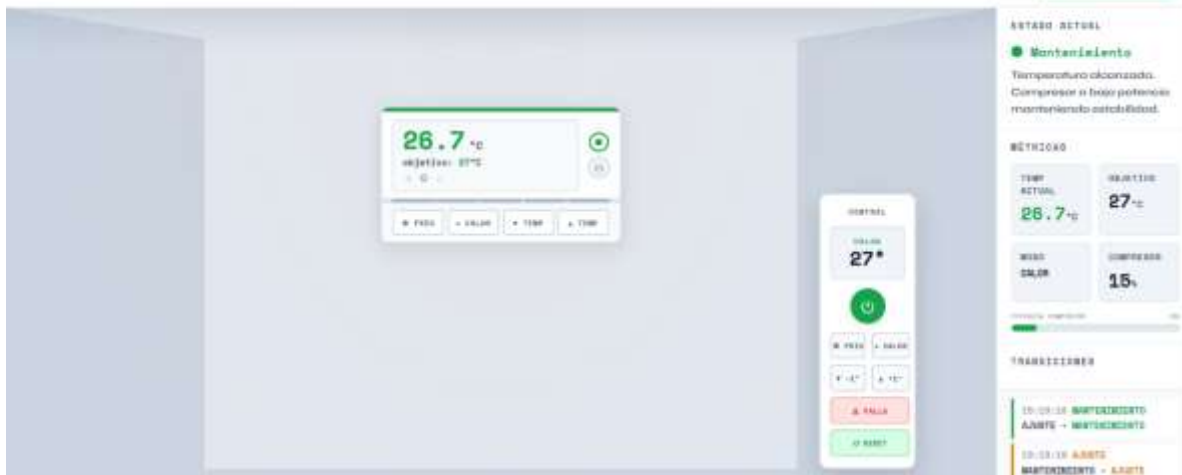
    @abstractmethod
    def apagar(self) -> None: pass

    @abstractmethod
    def set_modos(self, modos: str) -> None: pass

    @abstractmethod
    def monitorear(self) -> None: pass

    @abstractmethod
    def reset(self) -> None: pass

    def _log(self, etiqueta: str, mensaje: str) -> None:
        print(f"[{etiqueta}] {mensaje}")
```



Conclusión

El desarrollo del simulador de aire acondicionado permitió comprender de manera práctica. La aplicación del patrón de diseño estado en la construcción de sistemas dinámicos y adaptivos, la implementación demostró que el uso del patrón estado facilita su mantenimiento, escalabilidad y compresión, resultó ser una solución adecuada para representar sistemas cuyo comportamiento depende de condiciones cambiantes, evitando estructuras complejas de control como múltiples condicionales

Referencias Bibliográficas

- Design Patterns: Elements of Reusable Object-Oriented Software
Erich Gamma, Richard Helm, Ralph Johnson y John Vlissides.
Addison-Wesley, 1994.
(Libro base donde se define el patrón Estado y otros patrones de diseño.)
- Head First Design Patterns – Eric Freeman y Elisabeth Robson.
O'Reilly Media, 2004.
(Explica el patrón Estado de forma práctica y fácil de entender.)